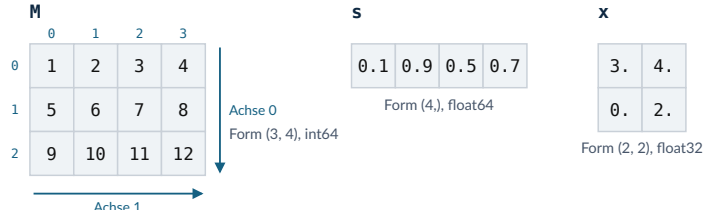
NumPy: Arrays, Formen und Broadcasting

M2 · Data-Science-Werkzeugkasten

Das Array: ein Block mit Beschreibung

Ein `ndarray` besteht aus einem Datenblock im Speicher mit genau einem Datentyp für alle Einträge und einer Beschreibung (Form, Typ, Abstände), wie er zu lesen ist. Rechnen passiert in kompiliertem Code; Sichten wie `M.T` ändern nur die Beschreibung. Die Beispiele dieser Karte verwenden drei kleine Arrays: `M = np.arange(1, 13).reshape(3, 4)` mit den Werten 1 bis 12 (Form `(3, 4)`, `int64`), `s = np.array([0.1, 0.9, 0.5, 0.7])` (Form `(4,)`, `float64`) und `x = np.array([[3., 4.], [0., 2.]], dtype=np.float32)` (Form `(2, 2)`).



Zugriff: vor dem Komma die Zeilen (Achse 0), dahinter die Spalten (Achse 1).
M[1, 2] ist 7, M[1] die Zeile [5 6 7 8], M[:, 2] die Spalte [3 7 11].
s hat eine Achse mit vier Einträgen, s[1] ist 0.9.
Die Form zählt die Einträge je Achse, der Datentyp gilt für alle Einträge.

Attribut	Bedeutung	Bei `M`	Bei den Kursdaten `E`
`shape`	Form: ein Tupel mit der Anzahl der Einträge je Achse	`(3, 4)`	`(400, 384)`
`ndim`	Anzahl der Achsen, also der Einträge von `shape`	`2`	`2`
`dtype`	Datentyp aller Einträge	`int64`	`float32`
`itemsize`, `size`, `nbytes`	Bytes je Eintrag, Anzahl der Einträge, Bytes insgesamt	`8`, `12`, `96`	`4`, `153600`, `614400`
`strides`	Abstand in Bytes zum nächsten Eintrag, je Achse einer	`(32, 8)`	`(1536, 4)`

Formen sind Tupel: `(4,)` ist eine Achse mit vier Einträgen, `()` eine einzelne Zahl. Die Angaben gelten für NumPy ab 2.2 auf 64-Bit-Systemen.

Arrays anlegen

`np.array([[3., 4.], [0., 2.]], dtype=np.float32)`	
`np.array([1, 2.5])`	ein Typ je Array: ganz float64
`np.zeros((2, 3))`	Nullen, float64; dtype= wählbar
`np.ones((2, 3))`	Einsen, ebenso
`np.arange(6)`	[0 1 2 3 4 5], ganze Zahlen int64
`np.linspace(0, 1, 5)`	[0. 0.25 0.5 0.75 1.]
`np.save("x.npy", x)`	Werte, Form und Typ; np.load lädt zurück
`rng = np.random.default_rng(42)`	
`rng.standard_normal((2, 3))`	Normalverteilung, float64
`M.astype(np.float32)`	anderer Typ, neues Array

Zugriff und Slicing

`M[0]`	Zeile 0: [1 2 3 4], Form (4,)
`M[0:1]`	Zeile 0 als Slice: [[1 2 3 4]], Form (1, 4)
`M[0, 2]`	ein Eintrag: 3, Form ()
`M[:, 1]`	Spalte 1: [2 6 10], Form (3,)
`M[1:, :2]`	[[5 6] [9 10]], Form (2, 2)
`M[::2]`	jede zweite Zeile, (2, 4); M[-1]: die letzte

Vor dem Komma die Zeilen, dahinter die Spalten, `:` heißt „alle“. Ein Index (`M[0]`) entfernt die Achse, ein Slice (`M[0:1]`) behält sie; ein Slice ist eine Sicht auf denselben Datenblock.

Reduktionen und Achsen

`M.sum()`	78, Form ()
`M.sum(axis=0)`	[15 18 21 24]: je Spalte, Form (4,)
`M.sum(axis=1)`	[10 26 42]: je Zeile, Form (3,)
`M.sum(axis=1, keepdims=True)`	[[10] [26] [42]], Form (3, 1)
`M.max(axis=1)`	[4 8 12]; M.argmax(axis=1): [3 3 3]
`M.mean(axis=1)`	[2.5 6.5 10.5]; M.std(axis=1): [1.118 ...]
`(M > 6).any(axis=1)`	[False True True]; .all: [False False True]
`M.cumsum(axis=1)`	laufende Summe, (3, 4); ohne axis flach (12,)
`np.linalg.norm(M, axis=1)`	Länge je Zeile: Wurzel der Quadratsumme

Merkregel: Die bei `axis=` genannte Achse wird zusammengefasst und verschwindet aus der Form (`axis=-1`: die letzte Achse); mit `keepdims=True` bleibt sie mit Länge 1.

Formen ändern, Zusammenfügen und Aufteilen

`M.T`	(3, 4) -> (4, 3), Sicht; v = M[0] hat (4,)
`M.reshape(2, 6)`	gleiche Einträge in Zeilenreihenfolge
`M.reshape(-1)`	flach, (12,): -1 rechnet die fehlende Anzahl aus
`v[None, :]`	(4,) -> (1, 4): Achse vorn einfügen
`v[:, None]`	(4,) -> (4, 1): Achse hinten einfügen
`np.concatenate([M, M], axis=0)`	(6, 4); gleiche Form außer auf der Achse
`np.stack([M, M])`	neue Achse vorn: (2, 3, 4); alle gleich geformt
`np.split(M, [1], axis=0)`	Teile (1, 4) und (2, 4), Sichten

`reshape` behält die Anzahl der Einträge: Aus 12 werden `(2, 6)`, `(4, 3)` oder `(12,)`, nicht `(5, 2)`. `M.reshape(4, 3)` und `M.T` haben beide die Form `(4, 3)`, aber andere Werte: `reshape` füllt die neue Form zeilenweise mit 1, 2, 3, ..., `.T` vertauscht Zeilen und Spalten. Zusammenfügen legt ein neues Array an, `split` liefert Sichten.

Datentypen

Typ	Bytes	Einsatz
`bool`	1	Masken
`int8` bis `int64`	1–8	ganze Zahlen; Positionen sind `int64`
`uint8`	1	0 bis 255, Bilddaten
`float16`, `bfloat16`	2	halbe Genauigkeit; `bfloat16` nur in PyTorch
`float32`	4	KI-Standard, rund 7 Stellen
`float64`	8	Python-`float`, rund 16 Stellen, Standard von `np.array([1.0])`

Ganzzahlarrays laufen ohne Warnung über (`int8`: 127 + 1 ergibt −128). Ein Python-`float` lässt `float32` bei `float32` (`x + 1.0`), hebt Ganzzahlarrays aber auf `float64`; NumPy-Typen folgen Promotion-Regeln (`int32 + float32 → float64`). Grenzen: `np.finfo`, `np.iinfo`.

Hochschule Worms
University of Applied Sciences

Broadcasting

Formen werden rechtsbündig untereinandergelegt und von rechts verglichen. Zwei Achsen passen, wenn sie gleich viele Einträge haben oder eine die Länge 1 hat; die 1 wird gedehnt. Fehlende Achsen werden links als 1 ergänzt.

Ausdruck	Formen	Ergebnis
`M + 1.0`	`(3, 4)` gegen `()`	`(3, 4)`: die Zahl wird zu jedem Eintrag addiert
`M + v` mit `v = M[0]`	`(3, 4)` gegen `(4,)`	`(3, 4)`: `v` wird zu jeder Zeile addiert, Eintrag für Eintrag
`M / c[:, None]` mit `c = M.sum(axis=1)`	`(3, 4)` gegen `(3, 1)`	`(3, 4)`: jede Zeile wird durch ihren eigenen Wert geteilt
`M / c`	`(3, 4)` gegen `(3,)`	`ValueError`: 4 gegen 3 passt nicht
`A[:, None, :] - B[None, :, :]`	`(2, 1, 4)` gegen `(1, 3, 4)`	`(2, 3, 4)` für `A (2, 4)`, `B (3, 4)`: jede Zeile von A minus jede Zeile von B, alle Paare

Die stille Falle: `x / n` mit `n = np.linalg.norm(x, axis=1)` (Form `(2,)`, Werte `[5. 2.]`) läuft ohne Fehler und liefert `[[0.6 2.] [0. 1.]]` statt `[[0.6 0.8] [0. 1.]]`: geteilt wird spaltenweise. Deshalb die Achse ausdrücklich behalten: `keepdims=True` oder `n[:, None]`. Broadcasting vervielfacht die Eingaben nicht im Speicher, das Ergebnis braucht aber Speicher in seiner Ergebnisform.

Vektorisierung und ufuncs

`x * x`	elementweise, Form bleibt (2, 2)
`np.sqrt(x)`	auf jeden Eintrag; math.sqrt bricht ab
`x + 1.0`	bleibt float32: Python-Zahl passt sich an
`x + np.float64(1.0)`	float64: NumPy-Typen heben an

Rechenzeichen sind ufuncs: `x * x` ist `np.multiply(x, x)`.

Gruppe	Beispiele
Arithmetik	`+ - * / **` (`np.add`, `np.multiply`, ...)
Funktionen	`np.sqrt`, `np.exp`, `np.log`, `np.abs`
Vergleiche	`>=`, `==`, `np.isnan`
zwei Arrays elementweise	`np.maximum(a, b)`, `np.minimum(a, b)`: je Position, nicht die Reduktion `a.max()`
Logik auf Masken	`&` (und), der senkrechte Strich (oder), `~` (nicht)

Ausdrücke wie `x / n` erzeugen standardmäßig ein neues Ergebnisarray, jedes Zwischenergebnis einer Kette auch; `out=` und In-place-Schritte wie `x /= n` schreiben in vorhandene Arrays.

Matrixprodukt

`v @ v`	Skalarprodukt: 30, (4,) @ (4,) -> ()
`M @ v`	ein Skalarprodukt je Zeile: [30 70 110], (3,)
`M @ M.T`	Skalarprodukte aller Zeilenpaare, (3, 3)
`A @ B`	(m, d) @ (d, n) -> (m, n): d verschwindet
`M * v`	elementweise per Broadcasting, kein Skalarprodukt

Skalarprodukt: die Summe der Produkte der Einträge, `(a * b).sum()`; für Einheitsvektoren der Kosinus des Winkels zwischen ihnen. Beim Matrixprodukt muss die letzte Achse des linken Arrays zur vorletzten des rechten passen; steht rechts ein Vektor, zählt seine einzige Achse (`M @ v`).

Masken und Indexarrays

`mask = s >= 0.5`	[False True True True], Form wie s
`mask.sum()`	3; np.any(mask): True; np.all(mask): False
`s[mask]`	[0.9 0.5 0.7]: liest eine Kopie
`(s >= 0.3) & (s < 0.8)`	beide Bedingungen: [False False True True]
`s[np.array([2, 0])]`	Indexarray: [0.5 0.1], in dieser Reihenfolge
`np.sort(s)`	Werte: [0.1 0.5 0.7 0.9]
`np.argsort(s)`	Positionen: [0 2 3 1]
`i = np.argsort(-s, kind="stable")`	
`i[:2], s[i[:2]]`	die zwei höchsten: [1 3], [0.9 0.7]
`t = s.copy(); t[t < 0.5] = 0`	schreibt in t: [0. 0.9 0.5 0.7]

`argsort` liefert Positionen, keine Werte; `kind="stable"` hält Gleichstände in Eingabereihenfolge. Zwei Bedingungen verknüpft man mit `&` (und), dem senkrechten Strich (oder) und `~` (nicht), jede Bedingung in Klammern: `(s >= 0.3) & (s < 0.8)`. `and`, `or`, `not` funktionieren bei Arrays nicht. `if arr:` bricht mit `ValueError` ab, sobald das Array nicht genau einen Eintrag hat (auch wenn es leer ist); gemeint ist `np.any`, `np.all` oder `arr.size > 0`.

Sicht oder Kopie

Ausdruck	Ergebnis
Slice `M[1:]`, `M.T`, `reshape` wenn möglich	Sicht: gleicher Datenblock, Schreiben ändert das Original
`s[mask]`, `s[indices]` beim Lesen, `astype` (standardmäßig `copy=True`), `flatten` (flach wie `reshape(-1)`, immer Kopie), Ausdrücke wie `x / n`	Kopie oder neues Ergebnisarray: eigener Datenblock
`a.copy()`	Kopie, ausdrücklich
`x = x / n`	neues Array; `x` wird neu gebunden, das alte Array bleibt unverändert
`x /= n`	ändert das vorhandene Array; andere Namen und Sichten darauf sehen die Änderung

Prüfen: `np.shares_memory(a, b)`. Zeilenweise zusammenhängend: `a.flags["C_CONTIGUOUS"]`; `np.ascontiguousarray(a)` stellt es her, kopiert nur wenn nötig. `astype(np.float32, copy=False)` spart die Kopie nur, wenn der Typ schon passt.

Prüfen und Messen

`assert y.shape == (2, 2)`	erst die Form
`assert y.dtype == np.float32`	dann der Typ
`assert np.allclose(y, expected, atol=1e-6)`	
`np.array_equal(a, b)`	exakt gleich, auch die Form
`x_before = x.copy()`	Kopie VOR dem Aufruf
`%timeit f(x)`	Laufzeit, mehrere Durchläufe

Zuletzt die Werte mit Toleranz: `allclose` prüft absolut (`atol`) plus relativ (`rtol`, Standard `1e-5`). Form und Typ vorher prüfen: `np.allclose` broadcastet `(1, 2)` gegen `(2,)` ohne Fehler und meldet keinen Typunterschied, ein `float64`-Ergebnis besteht den Vergleich mit `float32`-Vorgaben. `x_before = x` wäre nur ein zweiter Name.

Das Suchmuster: Normalize · Compare · Select

`eps = np.float32(1e-6)`	Normalize: eps positiv, im Rechentyp
`norms = np.sqrt((x * x).sum(axis=1, keepdims=True) + eps)`	
`xn = x / norms`	x (N, D), norms (N, 1), xn (N, D)
`scores = (qn @ docs_n.T)[0]`	Compare: (1, D) @ (D, N) -> (N,)
`np.argsort(-scores, kind="stable")[:k]`	

Select: die letzte Zeile liefert die Positionen der bis zu k höchsten Scores (bei weniger Einträgen alle). `docs_n (N, D)` sind alle Dokumentzeilen so normalisiert, `qn (1, D)` die Anfrage; `[0]` entfernt die Achse der einen Anfrage. Die Länge einer normalisierten Zeile ist `‖x‖ / sqrt(‖x‖² + eps)`, nicht genau 1; die Scores sind stabilisierte Näherungen der Kosinus-Ähnlichkeit.